MLR Example - Problem 6.2 Predicting Software Reselling Profits

> (c) 2025 Galit Shmueli, Peter C. Bruce, Peter Gedeck
>
> Code included in
>
> *Machine Learning for Business Analytics: Concepts, Techniques, and Applications in Python* (Second Edition) Galit Shmueli, Peter C. Bruce, Peter Gedeck, and Nitin R. Patel. 2025.

Add comments to each **code** cell (no comments need to be added for Markdown).

Each code cell starts with two comments:

1. Your initials and last two digits of your ID number.
2. Provide a brief explanation of the code cell. If comments are already included in the code cell, you may revise them for clarity, summarize the existing comments, or leave them unchanged if they are already appropriate. For example,

JT88

This code cell imports necessary Python packages.

Tayko Software is a software catalog firm that sells games and educational software. It started out as a software manufacturer and then added third-party titles to its offerings. It recently revised its collection of items in a new catalog, which it mailed out to its customers. This mailing yielded 2000 purchases. Based on these data, Tayko wants to devise a model for predicting the spending amount that a purchasing customer will yield. The dataset **Tayko.csv** contains information on 2000 purchases. Table 6.12 describes the variables to be used in the problem (the dataset contains additional variables).

In [1]:
```
pip install mlba
```

```
Requirement already satisfied: mlba in /opt/anaconda3/lib/python3.13/site-pa
ckages (2.0.2)
Note: you may need to restart the kernel to use updated packages.
```

In [2]:
```
pip install mlxtend
```

```
Requirement already satisfied: mlxtend in /opt/anaconda3/lib/python3.13/site
-packages (0.23.4)
Requirement already satisfied: scipy>=1.2.1 in /opt/anaconda3/lib/python3.1
3/site-packages (from mlxtend) (1.15.3)
Requirement already satisfied: numpy>=1.16.2 in /opt/anaconda3/lib/python3.1
3/site-packages (from mlxtend) (2.1.3)
Requirement already satisfied: pandas>=0.24.2 in /opt/anaconda3/lib/python3.
13/site-packages (from mlxtend) (2.2.3)
Requirement already satisfied: scikit-learn>=1.3.1 in /opt/anaconda3/lib/pyt
hon3.13/site-packages (from mlxtend) (1.6.1)
Requirement already satisfied: matplotlib>=3.0.0 in /opt/anaconda3/lib/pytho
n3.13/site-packages (from mlxtend) (3.10.0)
Requirement already satisfied: joblib>=0.13.2 in /opt/anaconda3/lib/python3.
13/site-packages (from mlxtend) (1.4.2)
Requirement already satisfied: contourpy>=1.0.1 in /opt/anaconda3/lib/python
3.13/site-packages (from matplotlib>=3.0.0->mlxtend) (1.3.1)
Requirement already satisfied: cycler>=0.10 in /opt/anaconda3/lib/python3.1
3/site-packages (from matplotlib>=3.0.0->mlxtend) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in /opt/anaconda3/lib/pytho
n3.13/site-packages (from matplotlib>=3.0.0->mlxtend) (4.55.3)
Requirement already satisfied: kiwisolver>=1.3.1 in /opt/anaconda3/lib/pytho
n3.13/site-packages (from matplotlib>=3.0.0->mlxtend) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /opt/anaconda3/lib/python
3.13/site-packages (from matplotlib>=3.0.0->mlxtend) (24.2)
Requirement already satisfied: pillow>=8 in /opt/anaconda3/lib/python3.13/si
te-packages (from matplotlib>=3.0.0->mlxtend) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in /opt/anaconda3/lib/python
3.13/site-packages (from matplotlib>=3.0.0->mlxtend) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in /opt/anaconda3/lib/py
thon3.13/site-packages (from matplotlib>=3.0.0->mlxtend) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /opt/anaconda3/lib/python3.1
3/site-packages (from pandas>=0.24.2->mlxtend) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in /opt/anaconda3/lib/python3.
13/site-packages (from pandas>=0.24.2->mlxtend) (2025.2)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/lib/python3.13/sit
e-packages (from python-dateutil>=2.7->matplotlib>=3.0.0->mlxtend) (1.17.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in /opt/anaconda3/lib/py
thon3.13/site-packages (from scikit-learn>=1.3.1->mlxtend) (3.5.0)
Note: you may need to restart the kernel to use updated packages.
```

In [3]:
```
import matplotlib.pyplot as plt # for plotting
import mlba
import numpy as np # for numerical data handling
import pandas as pd # for tabular data handling
import statsmodels.formula.api as sm # for statistical modeling
from mlxtend.feature_selection import ExhaustiveFeatureSelector, SequentialF
from sklearn.linear_model import BayesianRidge, Lasso, LassoCV, LinearRegres
from sklearn.model_selection import cross_validate, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

```
no display found. Using non-interactive Agg backend
```

In [4]:
```
#VB67
#loading the dataset
```

```
tayko_df = mlba.load_data('Tayko.csv')
tayko_df.shape
```

Out[4]: (2000, 25)

In [37]:
```
#VB67
# Listing all column names in the DataFrame as a Python list
list(tayko_df.columns)
```

Out[37]:
```
['US',
 'Freq',
 'last_update_days_ago',
 'Web order',
 'Gender=male',
 'Address_is_res',
 'Spending']
```

In [8]:
```
#VB67
# Displaying the first few rows of the DataFrame tayko_df
tayko_df.head()
```

Out[8]:

	sequence_number	US	source_a	source_c	source_b	source_d	source_e	source
0	1	1	0	0	1	0	0	
1	2	1	0	0	0	0	1	
2	3	1	0	0	0	0	0	
3	4	1	0	1	0	0	0	
4	5	1	0	1	0	0	0	

5 rows × 25 columns

6.2.a Explore the spending amount by creating a pivot table for the categorical variables and computing the average and standard deviation of spending in each category.

In [10]:
```
#VB67
# The selected_var list contains the column names to retain in the DataFrame

selected_var = [
    'US', 'Freq', 'last_update_days_ago', 'Web order', 'Gender=male',
    'Address_is_res', 'Spending']
tayko_df = tayko_df[selected_var]
list(selected_var)
```

Out[10]:
```
['US',
 'Freq',
 'last_update_days_ago',
 'Web order',
 'Gender=male',
 'Address_is_res',
 'Spending']
```

In [11]:
```
#VB67
# Creating a pivot table summarizing spending by gender:
# Rows are grouped by 'Gender=male' (True/False)
# The spending values are aggregated using mean and standard deviation funct
# This gives insight into average spending and variability between male and

pd.pivot_table(tayko_df, index= 'Gender=male', values= 'Spending',
               aggfunc= ['mean', 'std'])
```

Out[11]:

	mean	std
	Spending	Spending
Gender=male		
0	107.339642	190.83233
1	98.350810	183.02006

In [12]:
```
#VB67
# Creating a pivot table to summarize spending amounts by 'Web order' status
# Groups rows by 'Web order' values (e.g., Yes/No)
# Aggregates the 'Spending' values by calculating both the mean and standarc
# This helps to understand average spending and variability based on whether

pd.pivot_table(tayko_df, index= 'Web order', values= 'Spending',
               aggfunc= ['mean', 'std'])
```

Out[12]:

	mean	std
	Spending	Spending
Web order		
0	82.902439	173.417088
1	129.199531	200.463840

In [13]:
```
#VB67

# Creating a pivot table to analyze spending based on whether the address is
# Groups data by 'Address_is_res' (e.g., True/False)
# Aggregates 'Spending' column using mean and standard deviation to summariz
# This helps understand spending behavior differences between residential ar

pd.pivot_table(tayko_df, index= 'Address_is_res', values= 'Spending',
               aggfunc= ['mean', 'std'])
```

Out[13]:

Address_is_res	mean Spending	std Spending
0	105.306162	199.521159
1	93.174208	132.204281

In [14]:

```
#VB67
# Creating a pivot table to summarize spending grouped by the 'US' column:
# Rows are grouped by the 'US' categorical variable (e.g., Yes/No)
# Aggregates the 'Spending' values using mean and standard deviation
# This helps to analyze average spending and its variability for US vs non-U

pd.pivot_table(tayko_df, index= 'US', values= 'Spending',
               aggfunc= ['mean', 'std'])
```

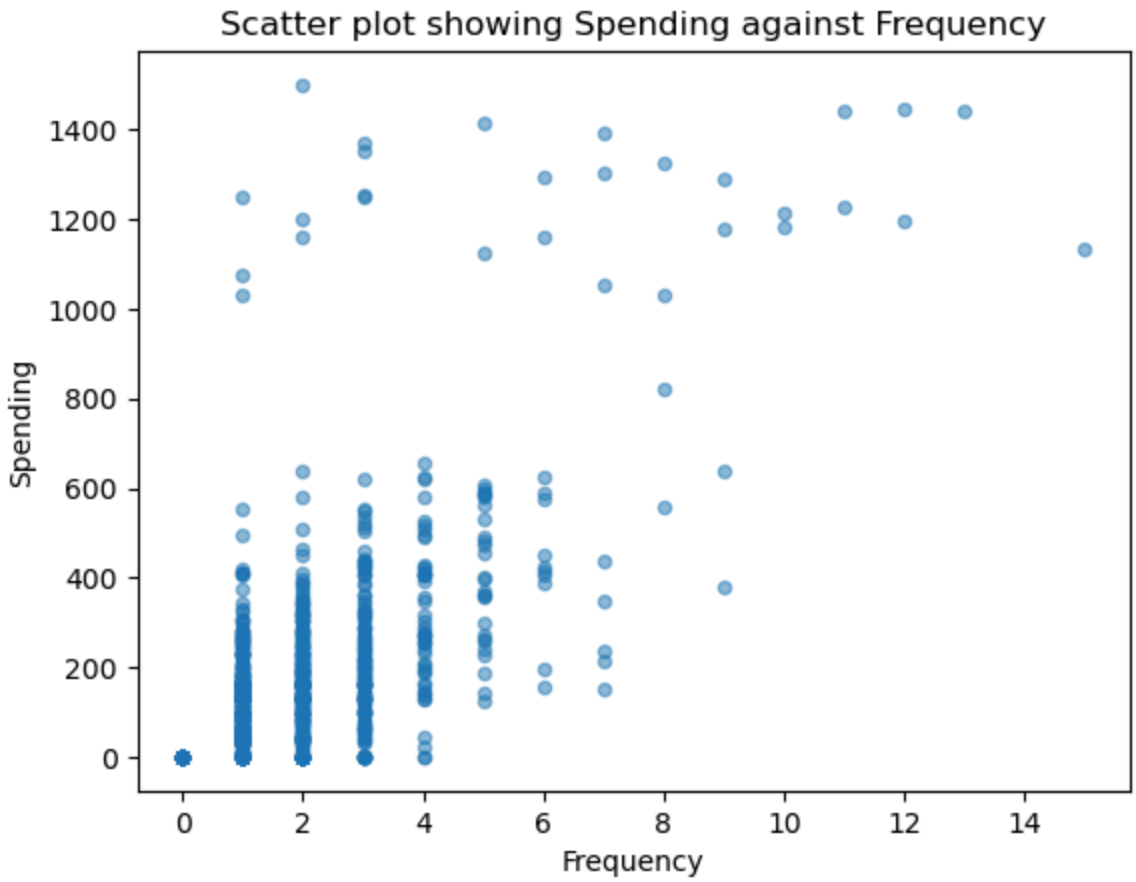
Out[14]:

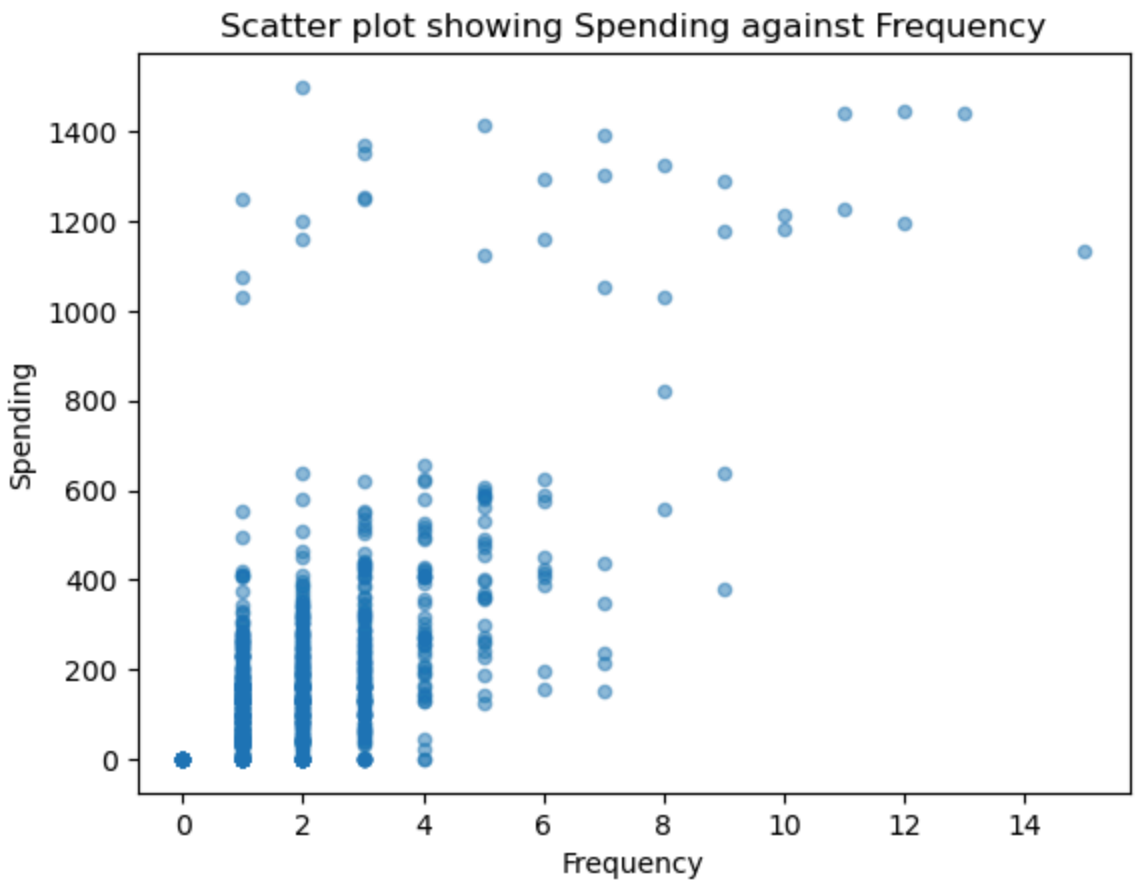
US	mean Spending	std Spending
0	101.216524	174.844401
1	102.924803	189.275664

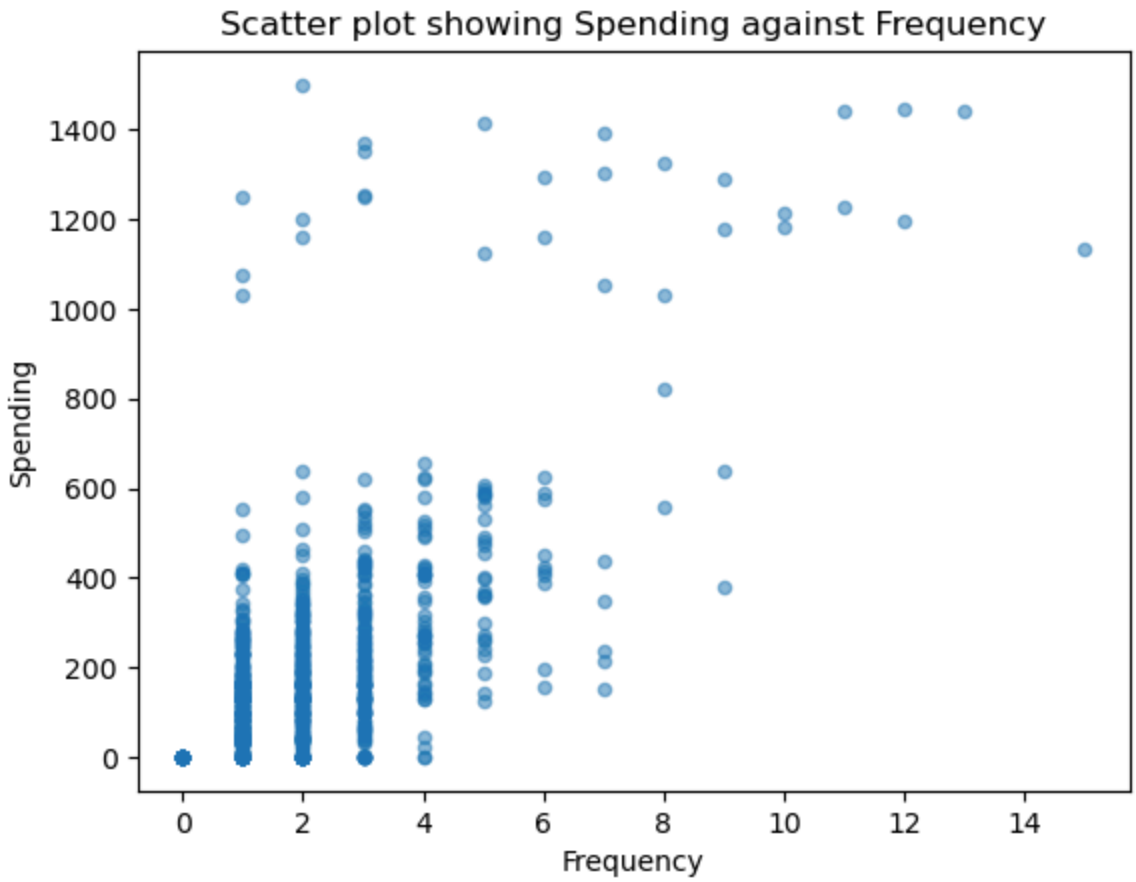
6.2.b.__ Explore the relationship between spending and each of the two continuous predictors by creating two scatterplots (Spending vs. Freq, and Spending vs. last_update_days_ago. Does there seem to be a linear relationship?

In [17]:

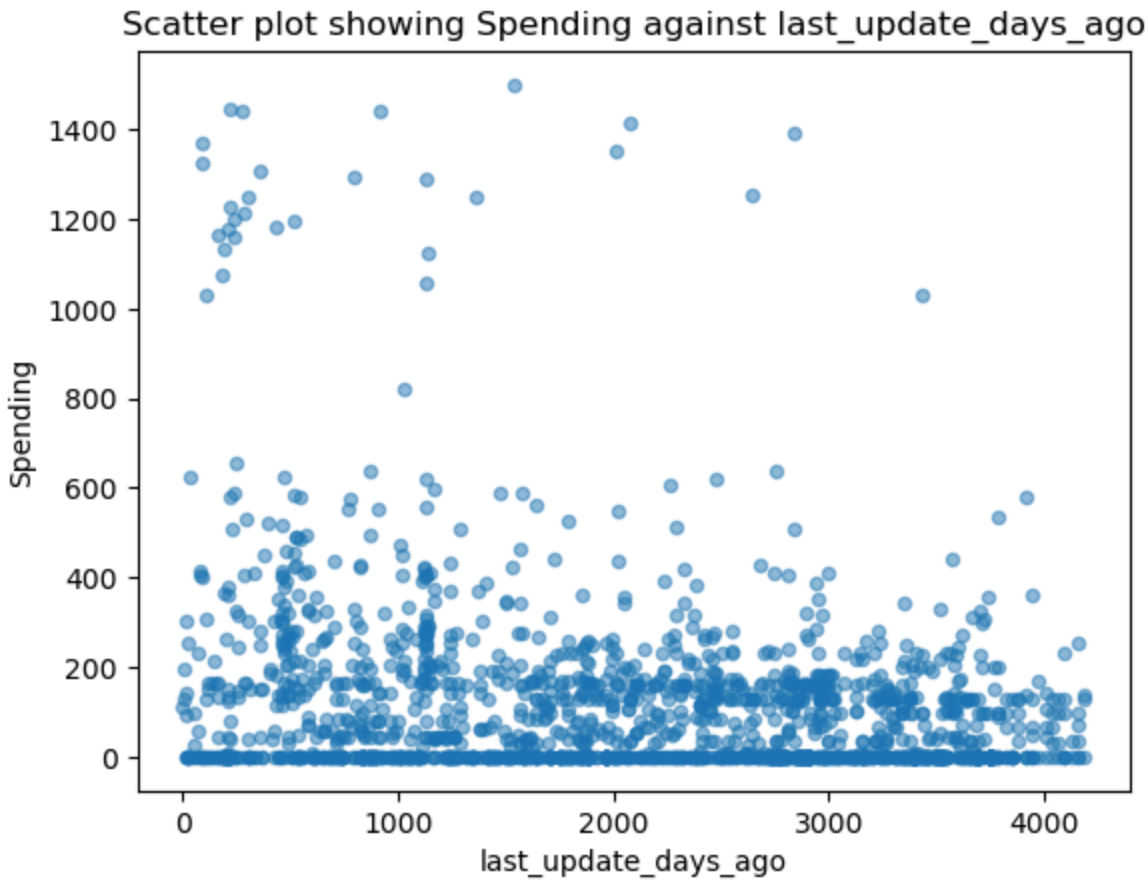
```
#VB67
# Generating a scatter plot to visualize the relationship between 'Freq' (fr
# 'Freq' is plotted on the x-axis and 'Spending' on the y-axis
# Alpha parameter set to 0.5 to make points semi-transparent, helping to see
# Set a descriptive title and axis labels for clarity
# Displaying the plot using plt.show()

%matplotlib inline

ax = tayko_df.plot.scatter(x='Freq', y='Spending', alpha=0.5)
ax.set_title('Scatter plot showing Spending against Frequency')
ax.set_ylabel('Spending')
ax.set_xlabel('Frequency')
plt.show()
```


Scatter plot showing Spending against Frequency
Spending
Frequency
0
200
400
600
800
1000
1200
1400
0
2
4
6
8
10
12
14


Scatter plot showing Spending against Frequency
Spending
Frequency
0
200
400
600
800
1000
1200
1400
0
2
4
6
8
10
12
14



In [20]:
```
#VB67
# Creating a scatter plot to examine the relationship between 'last_update_d
# Alpha set to 0.5 to make overlapping points semi-transparent for better vi
# Title and axis labels clearly describe the plot for interpretation


ax = tayko_df.plot.scatter(x='last_update_days_ago', y='Spending', alpha=0.5
ax.set_title('Scatter plot showing Spending against last_update_days_ago')
ax.set_ylabel('Spending')
# - 'last_update_days_ago' is plotted on the x-axis, and 'Spending' on the y
ax.set_xlabel('last_update_days_ago')
# - Display the plot with plt.show()
plt.show()
```



6.2.c. To fit a predictive model for spending:

6.2.c.i. Partition the 2000 records into training and validation sets.

In [22]:
```
#VB67
# Define the list of predictor variables to be used as features in the model
predictors = ['US', 'Freq', 'last_update_days_ago', 'Web order', 'Gender=mal
# Define the outcome variable representing the target variable for predictic
outcome = 'Spending'
# Creating the feature matrix X by selecting the predictor columns from the
X = tayko_df[predictors]
# Creating the target vector y by selecting the outcome column from the Data
y = tayko_df[outcome]
```

In [24]:
```
#VB67
# Spliting the data into training and holdout sets (40% holdout), with a fix
# Printing the shapes of the training and holdout feature sets
train_X, holdout_X, train_y, holdout_y = train_test_split(X, y, test_size =
print('Training set:', train_X.shape, 'Holdout set:', holdout_X.shape)
```

```
Training set: (1200, 6) Holdout set: (800, 6)
```

6.2.c.ii. Run a multiple linear regression model for Spending vs. all six predictors. Give the estimated predictive equation.

In [25]:
```
#VB67
# Initialize and fit a linear regression model on the training data
```

```
tayko_lm = LinearRegression()
tayko_lm.fit(train_X, train_y)
print('Intercept', tayko_lm.intercept_)
# Printing the intercept and coefficients to understand the influence of eac
print(pd.DataFrame({'Predictor': X.columns, 'coefficient': tayko_lm.coef_}))
```

```
Intercept 10.176297414603013
              Predictor  coefficient
0                    US    -4.620293
1                  Freq    91.274450
2  last_update_days_ago    -0.010374
3             Web order    18.628731
4           Gender=male    -9.111366
5        Address_is_res   -75.815354
```

6.2.c.iii. Based on this model, what type of purchaser is most likely to spend a large amount of money?

A purchaser with frequent catalog transactions in the last year and who has purchased by Web order at least once. This is based on the signs of the coefficients and on their statistical significance (low p-values).

In [27]:
```
#VB67
# Fit an OLS regression model using an R-style formula on the training data

train_df = train_X.join(train_y)
predictors = train_X.columns
# The formula specifies Spending as the outcome and the listed predictors as
formula = ("Spending ~ US + Freq + last_update_days_ago + Q('Web order') "
           "+ Q('Gender=male') + Address_is_res")
tayko_lm = sm.ols(formula=formula, data=train_df).fit()
# Printing a summary of the regression results including coefficients, signi
print(tayko_lm.summary())
```

```
                            OLS Regression Results
==============================================================================
==
Dep. Variable:               Spending   R-squared:                       0.5
25
Model:                            OLS   Adj. R-squared:                  0.5
22
Method:                 Least Squares   F-statistic:                     21
9.3
Date:                Tue, 23 Sep 2025   Prob (F-statistic):          1.26e-1
88
Time:                        12:19:06   Log-Likelihood:                -750
6.3
No. Observations:                1200   AIC:                         1.503e+
04
Df Residuals:                    1193   BIC:                         1.506e+
04
Df Model:                           6
Covariance Type:            nonrobust
==============================================================================
============
                            coef    std err          t      P>|t|      [0.025
0.975]
------------------------------------------------------------------------------
------------
Intercept                10.1763     13.543      0.751      0.453     -16.395
36.747
US                       -4.6203      9.636     -0.479      0.632     -23.526
14.285
Freq                     91.2744      2.809     32.490      0.000      85.763
96.786
last_update_days_ago     -0.0104      0.003     -3.029      0.003      -0.017
-0.004
Q('Web order')           18.6287      7.526      2.475      0.013       3.863
33.394
Q('Gender=male')         -9.1114      7.318     -1.245      0.213     -23.468
5.245
Address_is_res          -75.8154      9.162     -8.275      0.000     -93.790
-57.841
==============================================================================
==
Omnibus:                      956.068   Durbin-Watson:                   1.9
89
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            32247.6
50
Skew:                           3.402   Prob(JB):                         0.
00
Kurtosis:                      27.468   Cond. No.                     1.01e+
04
==============================================================================
==

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is corre
ctly specified.
[2] The condition number is large, 1.01e+04. This might indicate that there
```

```
are
strong multicollinearity or other numerical problems.
```

6.2.c.iv. If we used backward elimination to reduce the number of predictors, which predictor would be dropped first from the model?

Answer:

Based on the above model, US has the highest p-value. Therefore backward elimination would drop "US" first from the model.

6.2.c.v. Show how the prediction and the prediction error are computed for the first purchase in the validation set.

In [28]:
```
#VB67
# Displaying the first row of the validation DataFrame to preview a single c
valid_df = holdout_X.join(holdout_y)
first_obs_valid = valid_df.head(1)
print(first_obs_valid)
```

```
     US  Freq  last_update_days_ago  Web order  Gender=male  Address_is_res
\
674   1     2                  1346          0            1               1

     Spending
674         0
```

In [36]:
```
#VB67
# Assigning example values to variables representing predictor features for
US = 1; Freq = 2; last_update_days_ago = 1346; Web_order = 0
Gender_male = 1; Address_is_res = 1
```

In [31]:
```
#VB67
# Calculating predicted Spending using the regression equation and example p
# Each term multiplies a coefficient by the corresponding feature value and

Spending = (10.176297
                + (-4.620293*US)
                + (91.274450*Freq)
                + (-0.010347*last_update_days_ago)
                + (18.628731*Web_order)
                + (-9.111366*Gender_male)
                + (-75.815354*Address_is_res))
print(Spending)
```

```
89.251122
```

In [33]:
```
#VB67
# Calculating and print the prediction error by comparing the predicted Spen
print('Prediction Error = ', Spending - 0)
```

```
Prediction Error =  89.251122
```

6.2.c.vi. Evaluate the predictive accuracy of the model by examining its performance on the validation set.

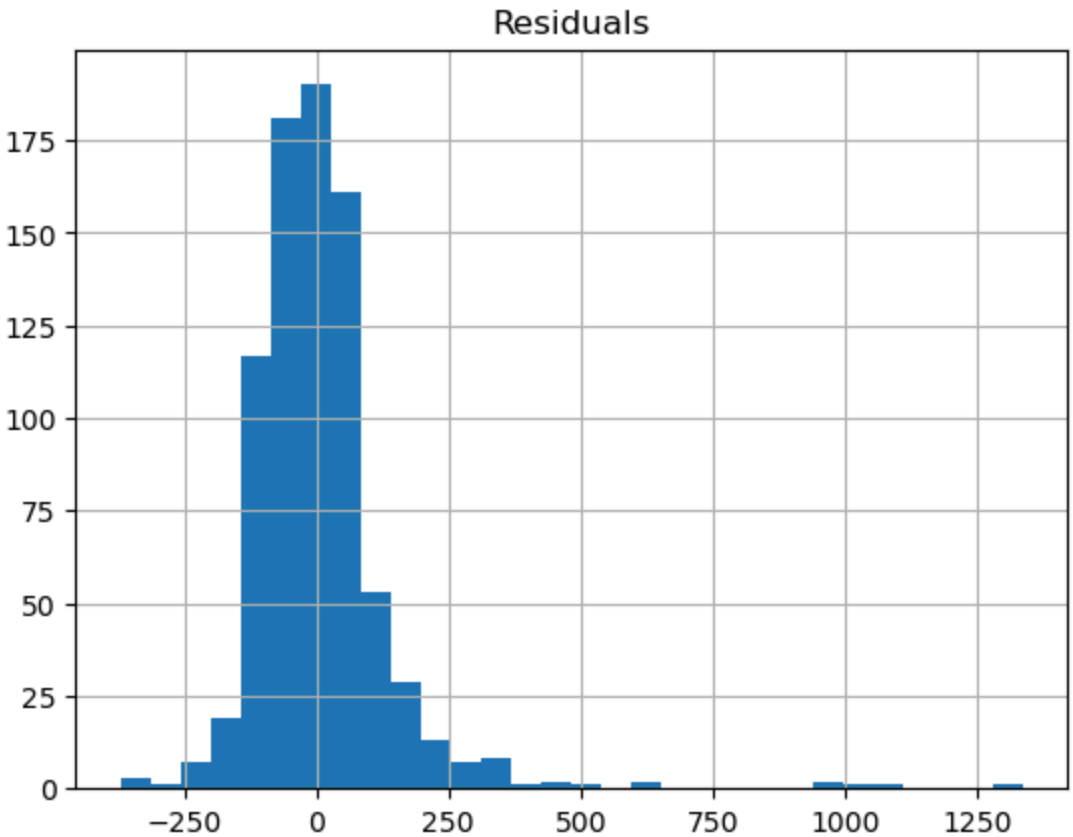
In [34]:
```
#VB67
# Evaluating and display model performance on training data using mlba's reg
print("\nModel performance on training data:")
mlba.regressionSummary(y_true=train_y, y_pred=tayko_lm.predict(train_X[predi

# Predicting spending on holdout data and create a DataFrame with predicted,
tayko_lm_pred = tayko_lm.predict(holdout_X[predictors])
result = pd.DataFrame({
    'Predicted': tayko_lm_pred,
    'Actual': holdout_y,
    'Residual': holdout_y - tayko_lm_pred,
})
# Show the first five rows of the prediction results
print("\nFirst few  predicted/actual values and residuals:\n")
print(result.head(5))

# Evaluating and display model performance on validation (holdout) data
print("\nModel performance on validation data:")
mlba.regressionSummary(y_true=holdout_y, y_pred=tayko_lm_pred)
```

```
Model performance on training data:

Regression statistics

Mean Error (ME) : 0.0000
Root Mean Squared Error (RMSE) : 125.9999
Mean Absolute Error (MAE) : 79.4772

First few  predicted/actual values and residuals:

       Predicted  Actual    Residual
674    89.214915       0  -89.214915
1699  202.231362     184  -18.231362
1282   49.159303       0  -49.159303
1315  824.841659    1289  464.158341
1210    0.121196       0   -0.121196

Model performance on validation data:

Regression statistics

Mean Error (ME) : 7.1933
Root Mean Squared Error (RMSE) : 136.7397
Mean Absolute Error (MAE) : 83.6010
```

6.2.c.vii. Create a histogram of the model residuals. Do they appear to follow a normal distribution? How does this affect the predictive performance of the model?

In [35]:
```
#VB67
# Calculating residuals (differences between actual and predicted values) on

all_residuals = holdout_y - tayko_lm_pred
pd.DataFrame({'Residuals': all_residuals}).hist(bins=30)
```

```
# Plot a histogram of residuals to visualize their distribution and assess m
plt.show()
```



In [ ]: